

Elevance

.NET Full-Stack Training Program

Patient Appointment & Health Records Portal

Sprint-Wise Project Plan

Domain: HealthAxis Health — Healthcare / Patient Care Management

Duration: 11 Weeks | 5 Sprints | Assessment Every Sprint

Project Overview

HealthAxis Health provides healthcare management services for patients dealing with chronic and behavioural health conditions. The current process for booking appointments and maintaining patient records is entirely manual — patients call the clinic, doctors use paper notes and spreadsheets, and there is no central system to track a patient's health history across multiple consultations.

This project requires the trainees to design and build a fully functional Patient Appointment & Health Records Portal — a web application that allows patients to register, search for doctors, and book appointments online, while giving doctors a digital workspace to manage their schedule and maintain accurate, accessible patient health records.

Technology Stack

Language: C# .NET 8

Backend API: ASP.NET Core Web API + ASP.NET MVC

ORM: Entity Framework Core (Code First) + EF 6.x

Frontend: Angular 17 + Blazor Server (Admin Console)

Database: MS SQL Server | MongoDB (reference / audit data)

Authentication: JWT (JSON Web Tokens) — Role-based: Patient, Doctor, Admin

Messaging: RabbitMQ via MassTransit — Event-Driven appointment notifications

Caching: Redis — Doctor availability and appointment slot caching

Logging: Serilog — Structured application logging

Monitoring: Kibana — Log visualisation and application monitoring

Testing: xUnit + Moq — Unit and integration tests (>80% coverage)

Deployment: AWS Elastic Beanstalk + AWS RDS + AWS S3

CI/CD: GitHub Actions (Workflows) + Jenkins

AI Tools: Windsurf — AI-assisted development

Version Control: Git + GitHub — feature branches, pull requests, code reviews

User Roles

Patient: Registers on the portal, searches for doctors by name or specialisation, books and manages appointments, receives real-time notifications, and views their complete health history.

Doctor: Logs in to view their daily and weekly schedule, accesses patient profiles before a consultation, and records diagnosis, treatment notes, and prescriptions after each visit.

Admin: Manages doctor profiles, specialisations, and clinic settings from a Blazor admin console, and monitors system activity and appointment reports.

Sprint Summary

High-level view of all five sprints aligned to the training program curriculum.

Sprint	Duration	Curriculum Focus	Deliverable
Sprint 1	Wk 1 – 2	Foundation, Git & C# OOP	Console-based appointment simulation with full OOP domain model App Review 1
Sprint 2	Wk 3 – 4	SQL Server, EF Core, MVC & WebAPI	ASP.NET CRUD API + MVC patient portal (patients, doctors, appointments, health records) App Review 2
Sprint 3	Wk 5 – 7	ASP.NET Core + Blazor + Angular	Secured Web API + Blazor admin console + Angular patient & doctor portal App Review 3
Sprint 4	Wk 8 – 9	Full Stack Integration	RabbitMQ messaging, Redis caching, Serilog + Kibana, unit & integration tests App Review 4
Sprint 5	Wk 10 – 11	Production & Deployment	Live AWS deployment + Jenkins + GitHub Actions CI/CD + Windsurf AI tools Final Assessment

Sprint 1 — Foundation, Git & C# OOP

Duration: Week 1 – Week 2 (2 weeks)

Curriculum: Dev Tools, Git & Web UI (Wk 1) | C# & OOP (Wk 2)

App Review 1: Viva + Coding — C# Console App (End of Week 2)

PS-02 Deliverable: A fully working console simulation of the appointment system built in pure C#

Learning Objectives

- Set up the full development environment: Visual Studio 2022, VS Code, SQL Server, Git, Node.js, Angular CLI.
- Understand Agile project delivery — sprints, daily stand-ups, retrospectives, Definition of Done.
- Use Git and GitHub for version control — branching, commits, pull requests, and code reviews.
- Learn HTML, CSS, and JavaScript (ES6+) fundamentals that underpin Angular and Blazor in later sprints.
- Apply the four pillars of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction in C#.
- Design and implement interfaces and abstract classes to enforce contracts across components.
- Use C# collections (List<T>, Dictionary, Queue) and LINQ to filter, sort, and query in-memory data.
- Handle exceptions using try/catch/finally and create meaningful custom exception types.
- Write basic xUnit tests with Arrange/Act/Assert to validate business logic.

What Trainees Will Build

A console-based simulation of the appointment portal using pure C# only — no database, no web framework, no UI. This models every entity and operation that the full application will need, and forms the domain foundation that all later sprints build on.

Patient class: Properties: PatientId, FullName, DateOfBirth, Gender, PhoneNumber, Email, InsuranceId, CreatedDate. Methods: GetAge() — calculates age from DateOfBirth. GetProfileSummary() — returns a formatted one-line summary of the patient.

Doctor class: Properties: DoctorId, FullName, Specialisation, YearsOfExperience, ConsultationFee, IsActive. Methods: IsAvailable(DateTime date) — checks if the doctor has capacity on that date. GetScheduleSummary() — returns upcoming appointments count.

Appointment class: Properties: AppointmentId, Patient (object), Doctor (object), ScheduledDate, TimeSlot, Status (enum: Pending / Confirmed / Cancelled / Completed), CancellationReason. Methods: Confirm(), Cancel(reason), Complete(), GetDetails() — formatted appointment summary.

HealthRecord class: Properties: RecordId, Patient (object), Doctor (object), VisitDate, Diagnosis, Prescription, Notes. Method: GetSummary() — returns a formatted single-line record summary.

IAppointmentService interface: Defines: BookAppointment(patient, doctor, date, slot), CancelAppointment(appointmentId, reason), GetAppointmentsByPatient(patientId), GetAppointmentsByDoctor(doctorId), GetUpcomingAppointments() — returns all future Confirmed appointments.

AppointmentService class: Implements IAppointmentService using in-memory List<Appointment> as data store. Throws custom exceptions: AppointmentConflictException (if a slot is already taken), PastDateException (if booking date is in the past), DoctorUnavailableException (if IsAvailable returns false). Uses LINQ to filter and sort appointments by date, status, and doctor.

HealthRecordService class: Methods: AddRecord(record), GetRecordsByPatient(patientId), GetRecordsByDoctor(doctorId). Uses LINQ to order records by VisitDate descending for the health history view.

Console menu application: A menu-driven console program with the following options: (1) Register a new patient, (2) Add a new doctor, (3) Search doctors by specialisation, (4) Book an appointment for a patient, (5) View all appointments for a patient, (6) Confirm or cancel an appointment, (7) Add a health record after a completed appointment, (8) View health history for a patient. All operations run against in-memory collections and the full flow works end-to-end.

xUnit test project: Unit tests for AppointmentService:
BookAppointment_ValidSlot_ShouldCreateAppointment,
BookAppointment_PastDate_ShouldThrowPastDateException,
BookAppointment_SlotAlreadyTaken_ShouldThrowConflictException,
CancelAppointment_ExistingId_ShouldUpdateStatusToCancelled,
GetAppointmentsByPatient_ShouldReturnOnlyPatientAppointments.